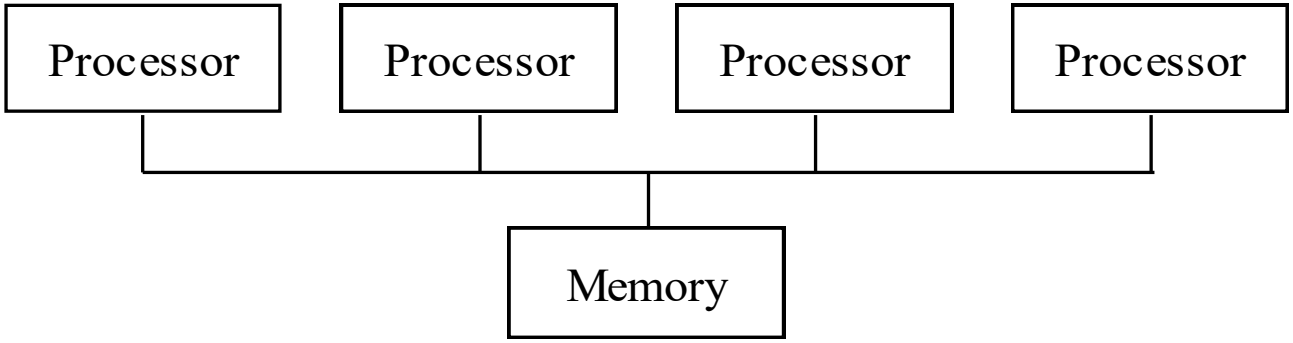
CPSC 375 High-Performance Computing

Lecture 16

Shared Memory Programming Using OpenMP

Shared-Memory Model



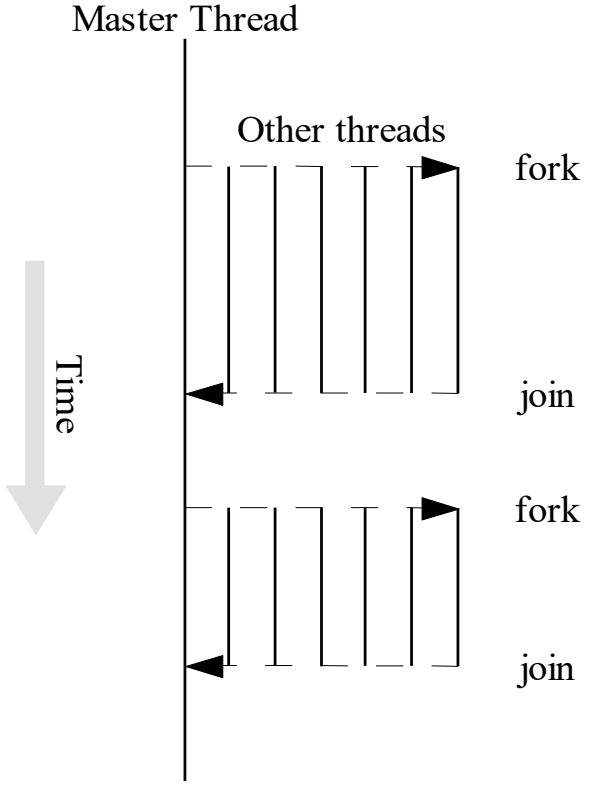
Processors interact and synchronize with each other through shared variables.

OpenMP

- OpenMP: An application programming interface (API) for parallel programming on multiprocessors
  - Compiler directives
  - Library of support functions
- OpenMP works in conjunction with Fortran, C, or C++
- C + OpenMP sufficient to program multiprocessors
- C + MPI + OpenMP a good way to program multicomputers built out of multiprocessors

Fork/Join Parallelism

- Initially only master thread is active
- Master thread executes sequential code
- Fork: Master thread creates or awakens additional threads to execute parallel code
- Join: At end of parallel code created threads die or are suspended



Shared-memory vs. Message-passing Model

- Shared-memory model
  - Only one active thread at start and finish of program
  - Changes dynamically during execution
- Message-passing model
  - All processes active throughout execution of program

Shared-memory vs. Message-passing Model

- Shared-memory model
  - Execute and profile sequential program
  - Incrementally make it parallel
  - Stop when further effort not warranted
- Message-passing model
  - Sequential-to-parallel transformation requires major effort
  - Transformation done in one giant step rather than many tiny steps

Parallel for Loops

- C programs often express data-parallel operations as **`for`** loops

```
for (i = 0; i < 10; i++)
    b[i] = i;
```

- OpenMP makes it easy to indicate when the iterations of a loop may execute in parallel
- Compiler takes care of generating code that forks/joins threads and allocates the iterations to threads

Pragmas

`#pragma omp` <*rest of pragma*>

- Compiler directive in C or C++
- Stands for "pragmatic information"
- A way for the programmer to communicate with the compiler

Parallel `for` Pragma

- Format:

```
#pragma omp parallel for
for (i = 0; i < 10; i++)
    b[i] = i;
```

- Compiler must be able to verify the run-time system will have information it needs to schedule loop iterations

Execution Context

- *Execution context*: address space containing all of the variables a thread may access
- Every thread has its own execution context
- Contents of execution context:
  - static variables
  - dynamically allocated data structures in the heap
  - variables on the run-time stack

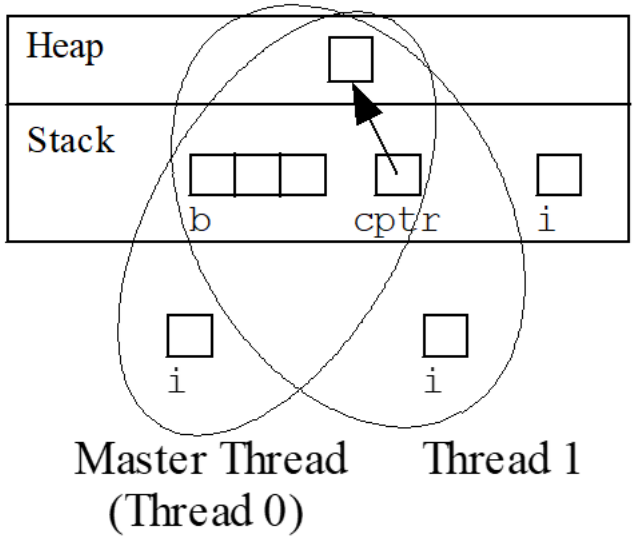
Shared and Private Variables

- *Shared variable*: has same address in execution context of every thread
- *Private variable*: has different address in execution context of every thread
- A thread cannot access the private variables of another thread

Shared vs Private Variables

```
int main()
{
    int b[3];
    char *cptr;
    int i;

    cptr = malloc(1);
    #pragma omp parallel for
    for (i = 0; i < 10; i++)
        b[i] = i;
}
```



`b`, `cptr`: shared

`i`: private

Runtime Functions for Thread Management

```
int omp_get_num_procs (void)
```

- Returns number of physical processors available for use by the parallel program

```
void omp_set_num_threads (int n)
```

- Sets the number of threads to **n**, to be active in parallel sections of code. May be called at multiple points in a program

```
int omp_get_thread_num()
```

- Returns the thread number, within the current team, of the calling thread.

Declaring Private Variables

`private (<variable list>)`

- Directs compiler to make one or more variables private

```
#pragma omp parallel for
for (i = 0; i < 10; i++)
     b[i] = i;
```

or

```
#pragma omp parallel private(i)
for (i = 0; i < 10; i++)
     b[i] = i;
```

Race Condition

- Consider this C program segment to compute π using the rectangle rule:

```
double area, pi, x;
int i, n;
...
area = 0.0;
for (i = 0; i < n; i++) {
   x = (i + 0.5)/n;
   area += 4.0/(1.0 + x*x);
}
pi = area / n;
```

Race Condition (cont.)

- If we simply parallelize the loop...

```
double area, pi, x;
int i, n;
...
area = 0.0;
#pragma omp parallel for private(x)
for (i = 0; i < n; i++) {
   x = (i + 0.5)/n;
   area += 4.0/(1.0 + x*x);
}
pi = area / n;
```

Critical Section

```
double area, pi, x;
int i, n;
...
area = 0.0;
#pragma omp parallel for private(x)
for (i = 0; i < n; i++) {
   x = (i + 0.5)/n;
#pragma omp critical
   area += 4.0/(1.0 + x*x);
}
pi = area / n;
```